Intermediate DevOps
Day 1: Continuous integration

United Airlines · Pluralsight instructor-led training

Nick Lotz

27–29 October 2026

About this course

- Instructor: Nick Lotz.
- About 30% of the time is lecture. About 70% is hands-on lab work.
- Each lecture prepares you for the lab that follows it.
- Ask questions at any time, by voice or in the chat.

The three days

Day	Lectures	Labs
1 · Continuous integration	DevOps foundations, version control and CI, pipelines in Harness, testing and quality gates	01, 02, 03
2 · Infrastructure as code and credentials	IaC concepts, Terraform, IaC in pipelines, credential management	04, 05, 06
3 · Containers, Kubernetes and delivery	Containers, ECS and Fargate, Kubernetes, continuous delivery	07, 08, 09

The labs build on each other. The application and pipelines from Day 1 are used again on Days 2 and 3.

Today’s sessions

1. Welcome and course overview
2. Lecture: DevOps foundations
3. Lecture: Version control and continuous integration
4. Lab 01: Lab environment and Harness Open Source
5. Lecture: Pipelines and Harness
6. Lab 02: Your first CI pipeline
7. Lecture: Testing and quality gates
8. Lab 03: Quality gates, pull requests and integration tests
9. Summary and questions

How the labs work

- The labs are Markdown files in the course repository: `github.com/nicklotz/devops-demos`.
- Each step explains why before it shows a command. Read the explanation before you run anything.
- Checkpoints show what you should see. Troubleshooting notes cover the common problems.
- "Going further" tasks are optional, for when you finish early.
- If you fall behind, `scripts/catch-up.sh 04` brings your machine to the start of Lab 04.

Your lab machine and United's platform

Each of you has an Ubuntu 24.04 machine with 8 CPUs and 16 GB of memory. You install every tool yourself during the labs.

In the lab	What it does	At United
Harness Open Source	Git hosting, pull requests, pipelines	Harness platform
k3d (Kubernetes in Docker)	Runs containers in a cluster	Amazon EKS
Docker Compose and an ECS task definition file	Runs containers without Kubernetes	Amazon ECS with Fargate
Terraform	Infrastructure as code	AWS CDK shared library
Harness secrets, Kubernetes Secrets	Credentials	TSS

Where DevOps came from

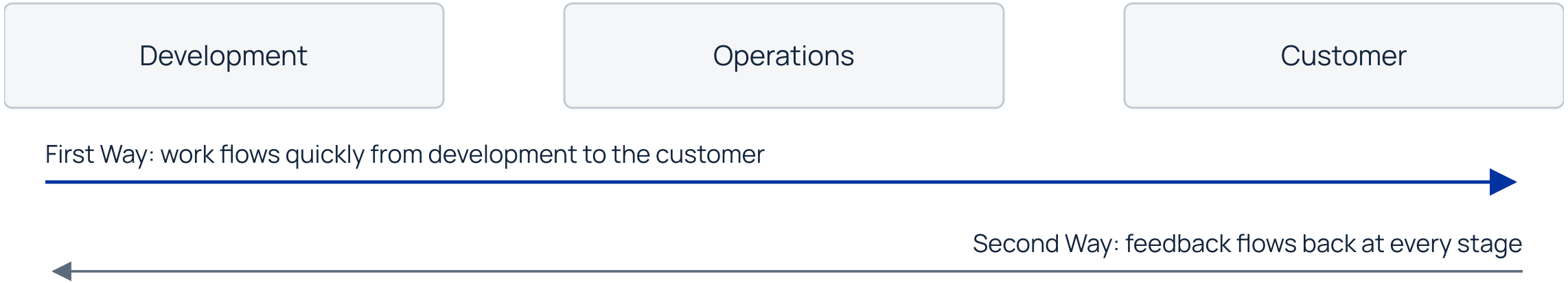
- 2008: Andrew Shafer and Patrick Debois meet at the Agile 2008 conference to discuss agile infrastructure.
- 2009: John Allspaw and Paul Hammond present "10+ Deploys per Day" about Flickr's development and operations teams.
- 2009: Debois organises the first devopsdays conference in Ghent, Belgium. The name DevOps comes from it.
- 2013 and 2016: *The Phoenix Project* and *The DevOps Handbook* describe the practices.
- From 2014: the State of DevOps research, later DORA, measures delivery performance.

The problem DevOps addresses



The two groups have opposite goals, and work moves between them in large, slow batches. DevOps practices give both groups one shared goal, which is frequent and safe change.

The Three Ways



Third Way: teams experiment, learn from failures, and improve their daily work

The Three Ways are the principles behind most DevOps practices (Kim et al., 2021).

Value streams and lead time



Work time: 6.5 hours

Lead time: about 9 days

Lead time for changes is the time from commit to running in production (Forsgren et al., 2018). In this example the change spends most of its lead time waiting.

Batch size and work in progress

- A batch is the amount of change released at one time.
- A large batch takes longer to test, fails more often, and is harder to debug when it fails.
- A small batch gives feedback sooner and is easier to roll back.
- Work in progress (WIP) is work that has started and not finished.
- Little's Law: average lead time = average WIP ÷ average throughput (Little, 1961).

Example: a team has 20 items in progress and finishes 5 per week. An item takes 4 weeks on average.

Measuring delivery performance

Metric	Question it answers	Measures
Deployment frequency	How often do we deploy to production?	Throughput
Lead time for changes	How long does a commit take to reach production?	Throughput
Change failure rate	What share of deployments cause a failure in production?	Stability
Failed deployment recovery time	How long does it take to recover from a failed deployment?	Stability

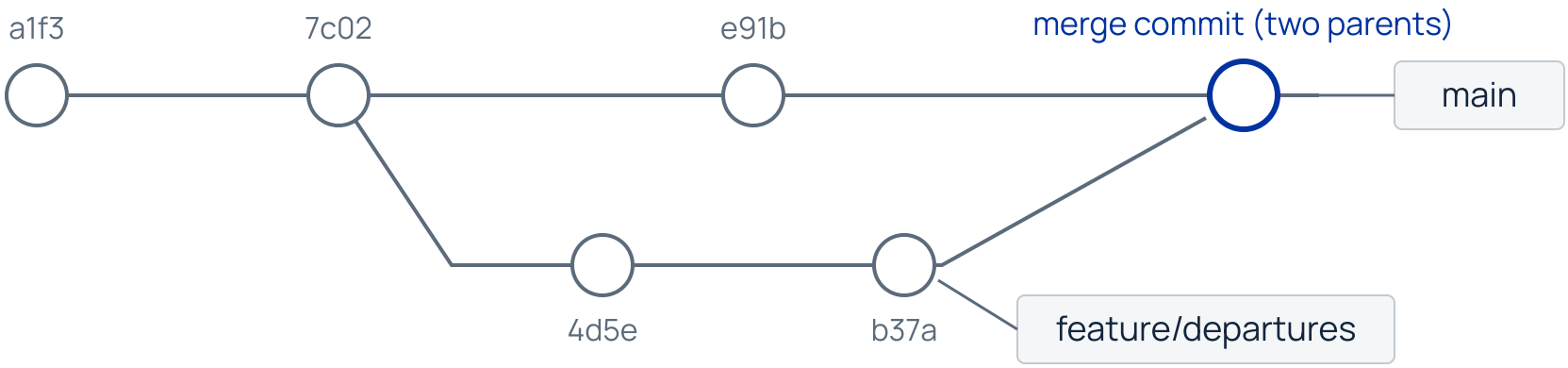
These are the DORA metrics (Forsgren et al., 2018). The 2024 DORA report adds a fifth measure, rework rate, which is the share of deployments that are unplanned fixes for production problems.

What the research found

- Teams with high throughput also had high stability. Speed and stability did not trade off against each other.
- Delivery performance predicted organisational performance, such as profitability and market share.
- Technical capabilities predicted delivery performance. Examples: version control, trunk-based development, continuous integration, test automation, deployment automation.
- This course practises several of these capabilities.

Source: Forsgren, Humble and Kim (2018), based on more than 23,000 survey responses.

What version control records



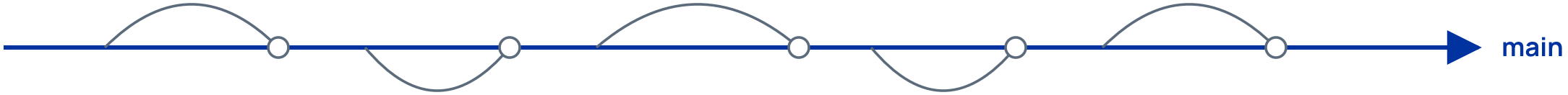
- A commit is a snapshot of all files, plus the author, time, message and parent commit.
- Git names each commit by a hash of its content and its parents. Changing anything changes the hash.
- A branch is a name that points to one commit. A merge commit has two parents.

Two branching strategies

	Git Flow (Driessen, 2010)	Trunk-based development
Long-lived branches	`main`, `develop`, release and hotfix branches	`main` only
Feature branches	Last until the feature is finished, often weeks	Last for hours, at most one or two days
When work is integrated	When a feature branch merges	At least once a day
Releases	Prepared on a release branch	Made from `main` at any time

The labs use trunk-based development with short-lived branches and pull requests.

Trunk-based development



Each branch has one author and lasts for hours, at most one or two days.

- Developers merge small changes into `main` at least once a day.
- Unfinished features are hidden behind feature flags, so `main` can always be released.
- High performers had three or fewer active branches, branches that lived less than a day, and no code freezes (Forsgren et al., 2018).

Pull requests and code review

- A pull request asks to merge one branch into another. It shows the diff, the discussion and the results of automated checks.
- Reviewers check correctness, design, tests and readability.
- A study at Microsoft found that reviews found fewer defects than developers expected. Knowledge transfer and team awareness were major outcomes (Bacchelli and Bird, 2013).
- Small pull requests get faster and more careful reviews.

Continuous integration practices

- Keep everything needed to build the software in version control.
- Automate the build, and make the build test itself.
- Everyone pushes commits to the main branch every day.
- Every push to the main branch triggers a build.
- Fix a broken build immediately. Keep the build under about ten minutes.

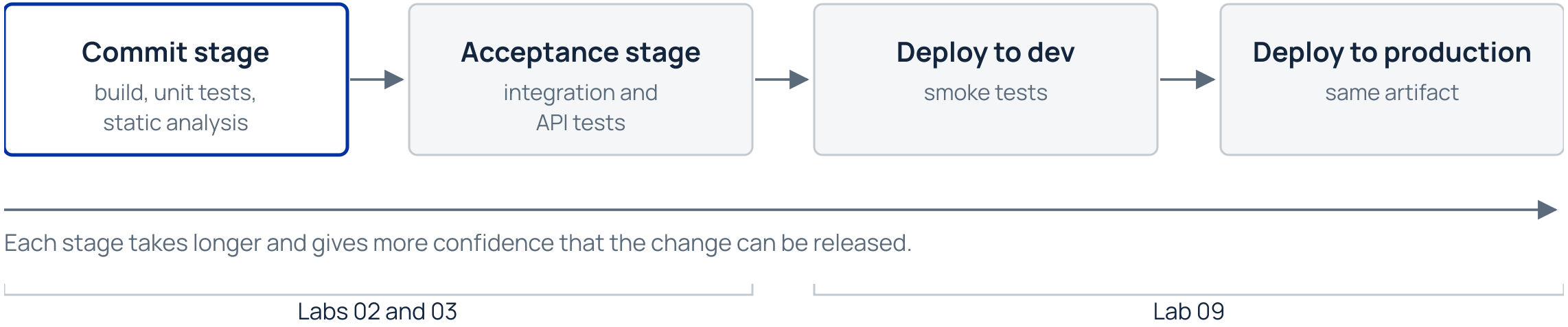
Fowler's article lists more practices. These five are the ones a pipeline enforces (Fowler, 2024).

Why integration delay is expensive



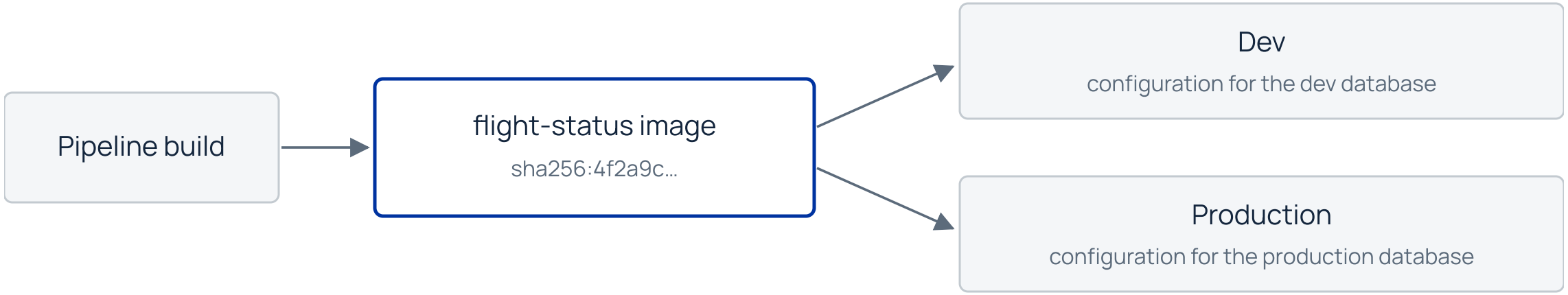
- The longer two branches stay apart, the more both of them change, and the larger the conflicts.
- A conflict found after three weeks involves code that its authors no longer remember well.
- Integrating every day keeps each merge small and each problem easy to locate.

The deployment pipeline



A change moves to the next stage only if it passed the previous one (Humble and Farley, 2010).

Build once, promote the same artifact



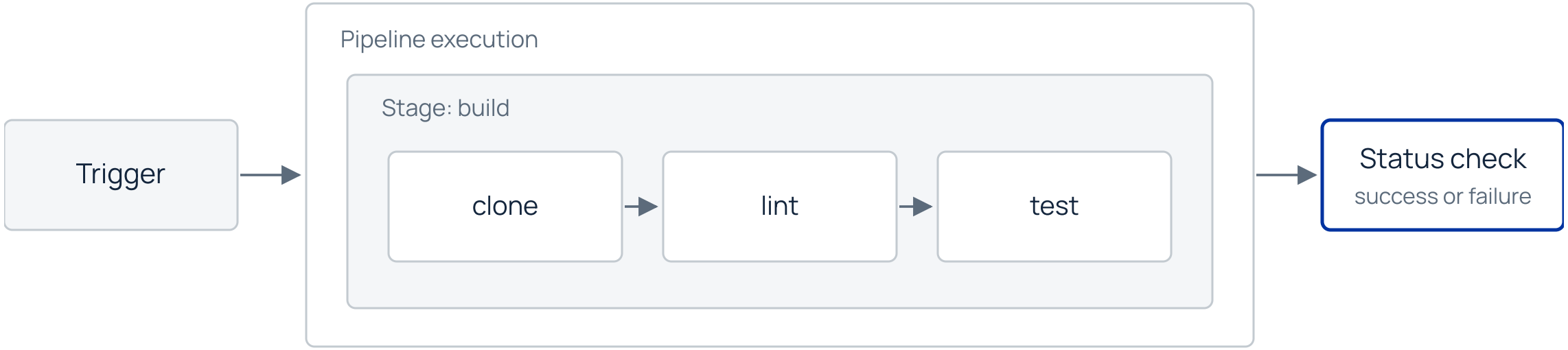
- Build the artifact once, in the pipeline. Deploy that same artifact to every environment.
- Only the configuration changes between environments. Keep configuration in the environment (Wiggins, 2011).
- If you rebuild for production, you deploy something that was never tested.

Lab 01: Lab environment and Harness Open Source

Open `labs/01-lab-environment.md` in the course repository.

- Install git, curl, jq, Docker and VS Code on your lab machine.
- Start Harness Open Source in a Docker container.
- Create the `flightops` project and the `flight-status` repository.
- Push the starter application and look at it in the Harness web interface.

The parts of a pipeline



- A trigger starts an execution: a push, a pull request, a tag, or a person selecting Run.
- A pipeline has stages. A stage has steps that run in order. A failed step stops the stage.
- The execution reports its status back to the commit and the pull request.

Pipeline as code

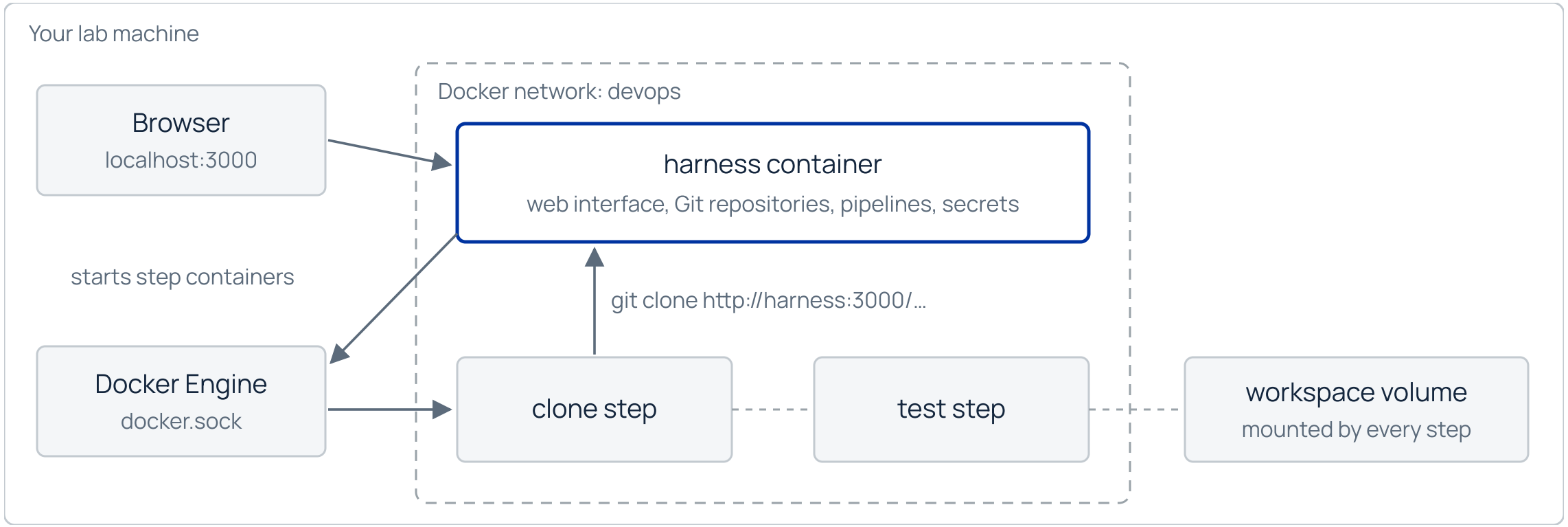
```
kind: pipeline
spec:
  stages:
  - name: build
    type: ci
    spec:
      steps:
      - name: test
        type: run
        spec:
          container: python:3.12-slim
          script: |
            pip install -r requirements-dev.txt
            pytest
```

- The pipeline is a YAML file in the repository: `.harness/ci.yaml`.
- A change to the pipeline is reviewed in a pull request, like a change to the code.
- The history shows who changed the pipeline, when, and why.

Containers as build environments

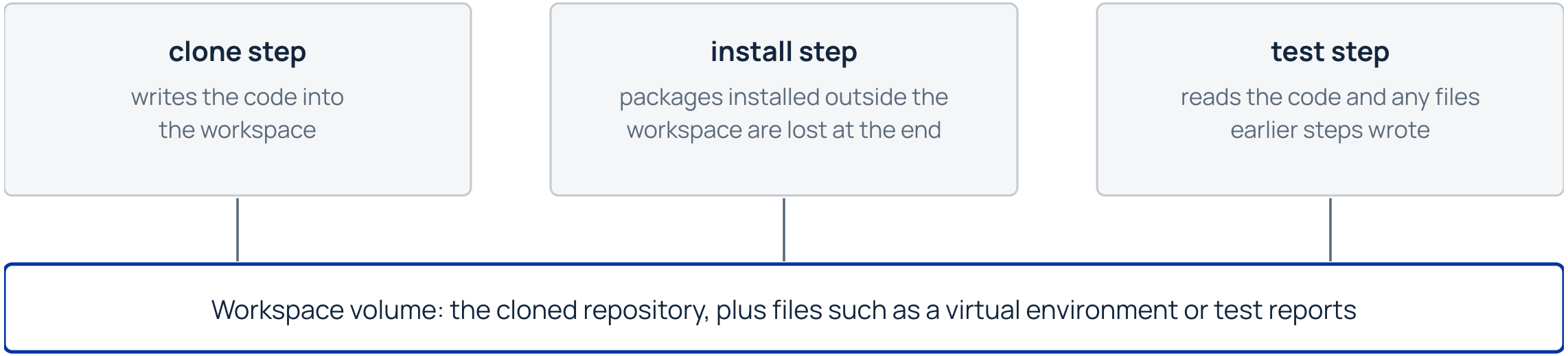
- Each step names a container image, for example `python:3.12-slim`.
- The image provides the tools. Nothing is installed on the machine that runs the pipeline.
- Every execution starts from the same image, so a result can be repeated.
- Pin the image tag. A floating tag such as `latest` can change between two runs.
- Steps can use different images: Python for tests, Terraform for infrastructure, kubectl for deployments.

How Harness Open Source runs a step on your machine



Harness asks Docker to start one container per step. The steps reach Harness by the name `harness` on the `devops` network.

Steps share the workspace



- Each step runs in a new container, with its own filesystem.
- Only the workspace is shared. To pass files to a later step, write them into the workspace.

Background services

```
- name: db
  type: background
  spec:
    container: postgres:17-alpine
    envs:
      POSTGRES_PASSWORD: test
- name: integration
  type: run
  spec:
    container: python:3.12-slim
    envs:
      DATABASE_URL: postgresql://postgres:test@db:5432/postgres
    script: pytest -m integration
```

- A background step starts a service and keeps it running while the later steps run.
- Later steps reach the service by the step name as host name: `db`.
- The service stops when the stage ends.

Triggers and events

Event	When it happens	Default trigger
Pull request created or reopened	A pull request is opened or reopened	Yes
Pull request branch updated	New commits are pushed to the branch of an open pull request	Yes
Branch updated	Commits are pushed to a branch, such as `main`	No. Add a trigger.
Tag created	A tag is pushed	No. Add a trigger.
Manual	Someone selects Run	Always available

Lab 02 adds a branch-updated trigger so that every push runs the pipeline, including the merge commits on `main`.

Conditions and expressions

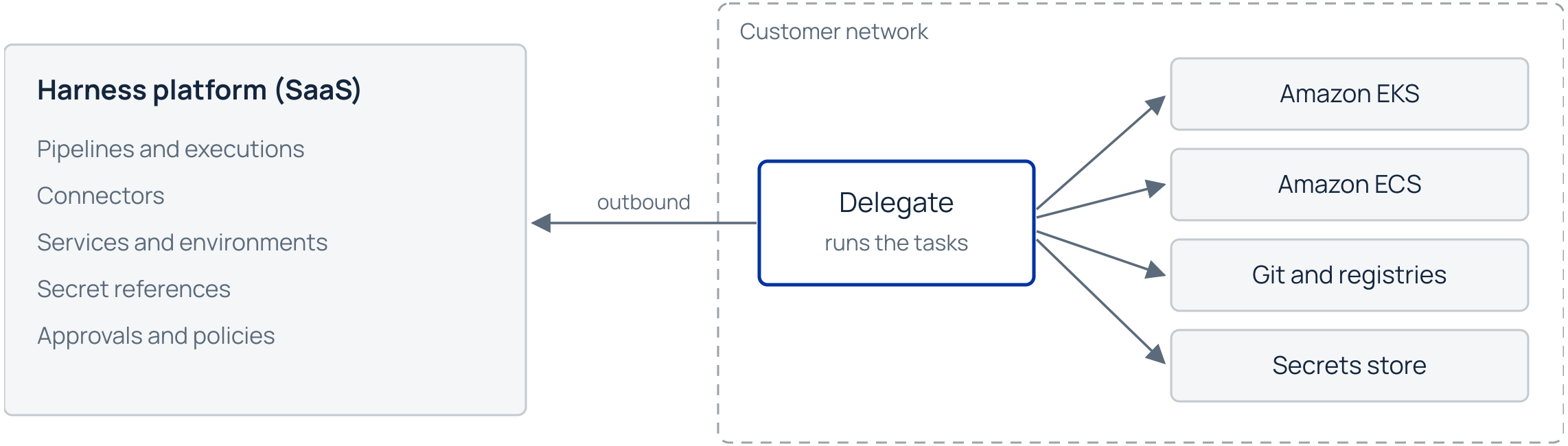
```
- name: report
  type: run
  when: build.event == "pull_request"
  spec:
    container: alpine:3.20
    script: |
      echo "Run ${{ build.number }}"
      echo "Commit ${{ build.commit }}"
- name: notify
  type: run
  when: failure()
  spec:
    container: alpine:3.20
    script: echo "An earlier step failed"
```

- `${{ … }}` inserts a value when the pipeline runs: the build number, commit, branch or event.
- `when:` decides at run time whether a step runs.
- `failure()` runs a step only after an earlier step failed. `always()` runs it in every case.

Harness Open Source and the Harness platform

Concept	Harness Open Source (lab)	Harness platform (United)
Where steps run	Containers on your lab machine	Harness Cloud machines, or a delegate in the company network
Hierarchy	Project, then repositories	Account, organization, project
Pipeline storage	`.harness/*.yaml` in the repository	In Harness, or in a Git repository
Expressions	`${{ build.number }}`	`<+pipeline.sequenceId>`
Deployments	Scripted in run steps	Deploy stages with services, environments and strategies

Harness platform building blocks



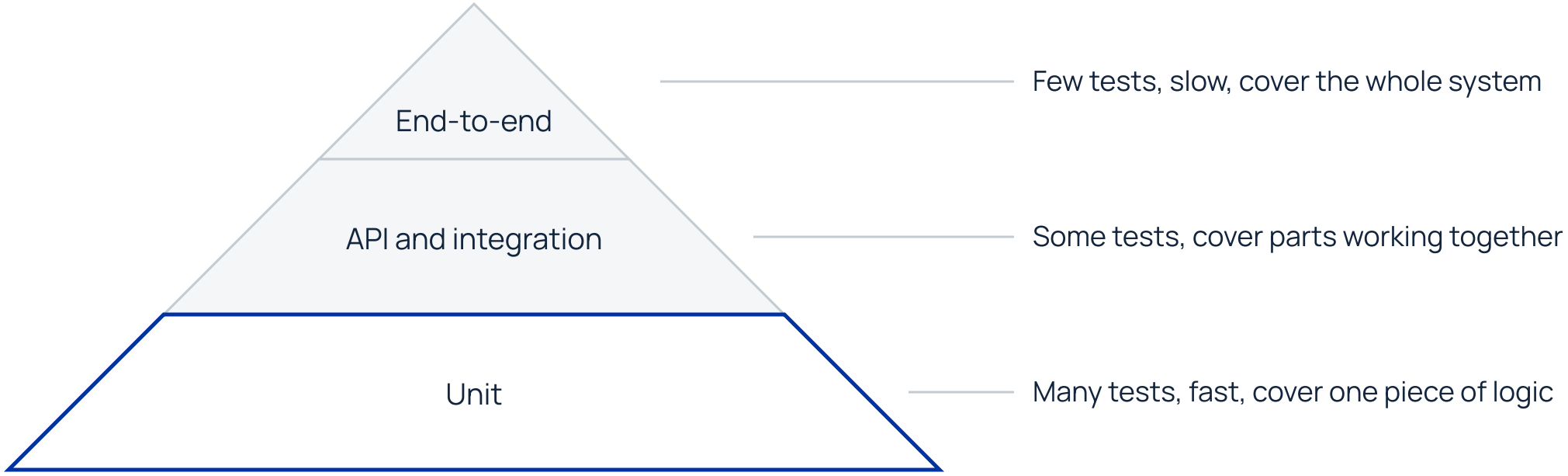
A delegate connects out to Harness and runs tasks inside the network. Connectors tell it how to reach each system.

Lab 02: Your first CI pipeline

Open `labs/02-first-ci-pipeline.md`.

- Run the tests in a container on your machine.
- Write `.harness/ci.yaml` with lint, unit test and API test steps, and run it in Harness.
- Read the logs of a failed run and fix the cause.
- Add a trigger so that every push runs the pipeline.

The test pyramid



Write many small, fast tests and a few slow, broad ones (Cohn, 2009; Vocke, 2018). Higher layers cost more to write, run and maintain.

Three kinds of tests in the flight-status app

Test	What it checks	What it needs	Time per test
Unit	Rules for flight status changes, input validation	Only the code	Milliseconds
API	HTTP endpoints, status codes and JSON responses	The app with in-memory storage	Tens of milliseconds
Integration	Reading and writing flights in a database	A running Postgres	Up to seconds

Lab 02 runs the unit and API tests. Lab 03 adds the integration tests, with Postgres as a background service.

Test doubles

Double	What it does
Dummy	Fills a parameter that the test does not use.
Stub	Returns fixed answers to calls.
Spy	A stub that also records how it was called.
Mock	Fails the test if the expected calls did not happen.
Fake	A working but simpler implementation, such as an in-memory database.

The flight-status API tests use a fake, an in-memory store, in place of Postgres (Meszaros, 2007).

Flaky tests

- A flaky test passes and fails on the same code.
- Common causes: timing and fixed waits, state shared between tests, test order, calls to external services.
- At Google, about 1.5% of test runs reported a flaky result, and almost 16% of tests showed some flakiness (Micco, 2016).
- Flaky tests teach people to ignore failures and re-run until the pipeline is green.
- Fix or quarantine a flaky test as soon as you find it.

What code coverage tells you

```
def can_board(status):
    return status in ("SCHEDULED", "DELAYED")

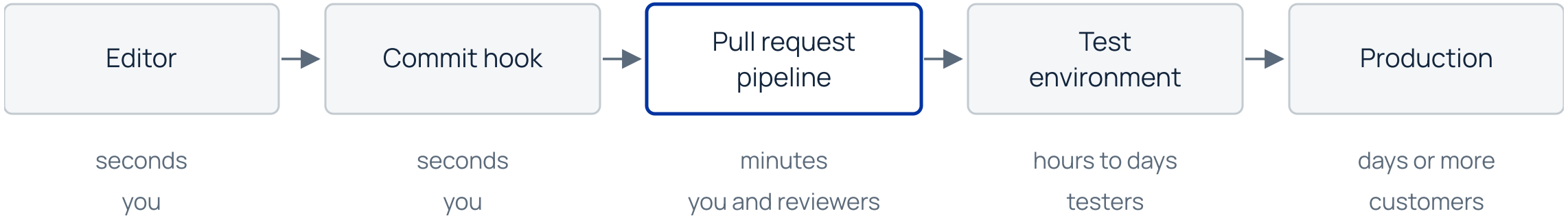
def test_can_board():
    can_board("SCHEDULED")      # runs the line, checks nothing
```

- Coverage measures which lines ran during the tests. This test reaches 100% coverage of `can_board`.
- Coverage does not measure whether the tests check the results.
- In five large Java programs, coverage had a low to moderate correlation with fault detection once the number of tests was held constant (Inozemtseva and Holmes, 2014).
- A coverage threshold is still useful as a floor that blocks large untested changes.

Static analysis and linting

- Static analysis reads the code without running it.
- Linters find unused imports, undefined names, risky patterns and style problems. The labs use ruff for Python.
- Type checkers find values of the wrong type before the code runs.
- Security analysis (SAST) finds patterns such as SQL built from user input.
- These checks take seconds, so the pipeline runs them first.

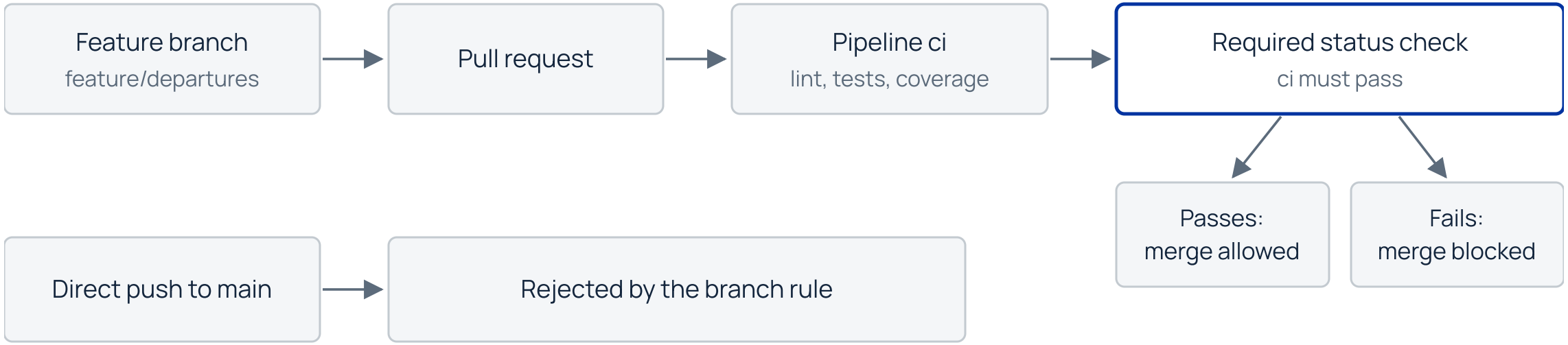
Finding problems earlier



The two lines under each step show how long it takes to find a problem, and who is affected.

"Shift left" means moving checks toward the start of this line. A problem found in the editor costs one person a few seconds. The same problem found in production affects customers.

Quality gates on the main branch



- A branch rule on `main` blocks direct pushes and requires a pull request.
- The rule names the `ci` status check. Merge is possible only after the latest run of `ci` passed.

Lab 03: Quality gates, pull requests and integration tests

Open `labs/03-quality-gates.md`.

- Add integration tests with Postgres as a background step, and a coverage threshold.
- Protect `main` with a branch rule and a required status check.
- Add a departures board on a branch and open a pull request.
- Fix what the tests and the pipeline find, then merge.

Day 1 summary

- DevOps aims at fast flow and fast feedback. The DORA metrics measure both.
- Continuous integration means small changes merged into `main` every day, each checked by an automated build.
- A pipeline is code. It is YAML in the repository, and each step runs in a container.
- Tests at several levels, static analysis and branch rules form the quality gates for `main`.
- Tomorrow covers infrastructure as code with Terraform, and how to handle credentials.

References

- Allspaw, J., and Hammond, P. (2009). 10+ Deploys per Day: Dev and Ops Cooperation at Flickr. Velocity Conference.
- Bacchelli, A., and Bird, C. (2013). Expectations, outcomes, and challenges of modern code review. ICSE 2013.
- Chacon, S., and Straub, B. (2014). *Pro Git*, 2nd ed. Apress. git-scm.com/book
- Cohn, M. (2009). *Succeeding with Agile*. Addison-Wesley.
- DORA (2024). *Accelerate State of DevOps Report*. Google Cloud. dora.dev
- Driessen, V. (2010). A successful Git branching model. nvie.com
- Forsgren, N., Humble, J., and Kim, G. (2018). *Accelerate*. IT Revolution.
- Fowler, M. (2024). Continuous Integration. martinfowler.com/articles/continuousIntegration.html
- Hammant, P. Trunk Based Development. trunkbaseddevelopment.com
- Harness. Harness Open Source and platform documentation. developer.harness.io
- Humble, J., and Farley, D. (2010). *Continuous Delivery*. Addison-Wesley.
- Inozemtseva, L., and Holmes, R. (2014). Coverage is not strongly correlated with test suite effectiveness. ICSE 2014.
- Kim, G., Humble, J., Debois, P., Willis, J., and Forsgren, N. (2021). *The DevOps Handbook*, 2nd ed. IT Revolution.
- Little, J. D. C. (1961). A proof for the queuing formula L = λW. *Operations Research* 9(3).
- Meszaros, G. (2007). *xUnit Test Patterns*. Addison-Wesley.
- Micco, J. (2016). Flaky Tests at Google and How We Mitigate Them. Google Testing Blog.
- Vocke, H. (2018). The Practical Test Pyramid. martinfowler.com
- Wiggins, A. (2011). The Twelve-Factor App. 12factor.net